Recette des postes EXPERTY

Documentation technique du code

Application de bureau Python / Tkinter - version 0.12

Document destiné au technicien ou au développeur qui reprend le projet : il explique comment le code est organisé, ce que fait chaque fichier, comment les données circulent et comment le faire évoluer sans rien casser. Chaque partie commence par une explication « en clair » avant d'entrer dans le détail technique.

Projet BTS SIO SLAM 3.1 - Suite du projet 1.4 FOG

Rédigé le 29/10/2026

Sommaire

1. Présentation du projet **4**

1.1 À quoi sert l'application . . . 4

1.2 Les règles de gestion à connaître (RG01 à RG10) . . . 4

1.3 Droits et prérequis . . . 4

2. Arborescence, lancement et fichiers produits **5**

2.1 Organisation des dossiers . . . 5

2.2 Lancer l'application . . . 5

2.3 Fichiers que l'application crée . . . 5

3. Architecture générale **6**

3.1 Les modules et leur rôle . . . 6

3.2 Qui appelle qui . . . 6

3.3 Les trois grands scénarios . . . 6

4. Les modules en détail **8**

4.1 main.py . . . 8

4.2 configuration.py . . . 8

4.3 profil.py . . . 8

4.4 catalogue.py . . . 8

4.5 powershell.py : la fonction unique . . . 9

4.6 sondes.py . . . 9

4.7 comparaison.py . . . 10

4.8 corrections.py . . . 11

4.9 session.py . . . 11

4.10 journal.py . . . 11

4.11 systeme.py . . . 12

4.12 pv.py . . . 12

4.13 interface.py . . . 12

5. Le thread de travail et la file de messages **14**

6. Le format du profil de référence **15**

7. Le catalogue des 25 contrôles **18**

8. La règle du verdict (RG03) **19**

9. Les 14 corrections automatiques **20**

10. Faire évoluer le projet **21**

10.1 Ajouter un profil . . . 21

10.2 Ajouter un contrôle (ordre à suivre) . . . 21

10.3 Règles à respecter quand on modifie le code . . . 21

10.4 Dépannage 21

11. Dépendances **22**

12. Procédure d'empaquetage (à compléter) **22**

13. Limites connues et points de vigilance **22**

14. Glossaire **23**

1. Présentation du projet

1.1 À quoi sert l'application

En clair : quand un poste vient d'être installé (par image FOG), personne ne vérifie qu'il est bien configuré. Cette application le fait : elle lit l'état réel du poste, le compare à un fichier de référence (le « profil »), affiche un tableau vert / rouge / orange, propose de corriger certains écarts, puis édite un procès-verbal (PV) PDF daté qui prouve le résultat.

C'est une **recette** : une vérification formelle, faite une fois, à la demande, sur le poste, avant sa remise au collaborateur. Ce n'est ni un diagnostic (chercher la cause d'une panne), ni une supervision (surveiller en continu).

L'application fait	L'application ne fait pas
Lit l'état du poste (25 contrôles)	Aucune surveillance continue, aucune exécution planifiée
Compare cet état au profil JSON choisi	Aucune action sur un autre poste (pas d'exécution à distance)
Corrige 14 types d'écarts, uniquement sur clic + confirmation	Ne désinstalle jamais un logiciel, ne supprime jamais un compte, n'active jamais BitLocker (RG08)
Journalise chaque correction (avant / après / durée)	N'arrête jamais le service Windows Update (RG09)
Édite un PV PDF avec la passe initiale et la passe finale	Pas de base de données, pas de signature électronique

1.2 Les règles de gestion à connaître (RG01 à RG10)

Le cahier des charges impose des règles qui expliquent beaucoup de choix du code. Voici où chacune est appliquée :

Règle	Principe	Où dans le code
RG01	Aucune correction sans clic « Corriger » puis confirmation en français	`interface._corriger` (askyesno avant le thread)
RG02	3 résultats possibles : Conforme, Écart, Indéterminé. Indéterminé n'est jamais « conforme »	`comparaison.evaluer_controle`
RG03	Verdict global selon la gravité des écarts (4 cas)	`comparaison.calculer_verdict`
RG04	Rien en dur : valeurs attendues dans le profil, environnement dans `recette.ini`	`profil.py`, `configuration.py`
RG05	Le profil est validé au chargement, erreur claire sans trace Python	`profil._valider`, `ErreurProfil`
RG06	Une correction se vérifie en relisant le contrôle, pas en croyant la commande	`corrections._executer`
RG07	Redémarrage ou nouvelle session annoncé : ligne rouge « en attente de prise en compte »	`ATTENTE` (corrections) et `interface._terminer_correction`
RG08	3 actions réservées à l'humain (désinstaller, BitLocker, supprimer un compte)	Aucune fonction de ce type n'existe ; ces contrôles sont « Manuelle » dans `catalogue.py`
RG09	Windows Update n'est jamais empêché : on le réactive	`corriger_maj_windows` (StartupType Manual)
RG10	Une correction lancée 2 fois ne fait aucun dégât (idempotence)	Début de `_executer` : si déjà conforme, on ne touche à rien

1.3 Droits et prérequis

- Windows 11 Pro avec PowerShell (`powershell.exe`) ; **droits d'administrateur** nécessaires pour la plupart des corrections (renommage, pare-feu, services, HKLM) et pour la sonde BitLocker. Le bandeau de la fenêtre affiche « Droits administrateur : oui / non ».
- Python 3 avec Tkinter (Tkinter est livré avec Python mais absent des distributions « embeddable »).
- Dépendances Python : voir la partie 11 (`requirements.txt`).
- **Toute exécution qui modifie le système se fait dans la machine virtuelle de test, jamais sur un poste réel.**

2. Arborescence, lancement et fichiers produits

2.1 Organisation des dossiers

```
racine du projet/            <- "dossier de l'application" (à côté de l'exécutable une fois empaqueté)
  recette.ini                configuration : dossiers, opérateur, délai des commandes
  requirements.txt           dépendances Python (versions figées)
  profils/                   1 fichier JSON par type de poste
    POSTE-COMPTABLE.json
    POSTE-ACCUEIL.json
    POSTE-DIRECTION.json
  pv/                        procès-verbaux PDF produits       (créé au besoin)
  journal/                   journal_AAAA-MM-JJ_HHMMSS.log + session_en_cours.json (créé au besoin)
  src/                       le code Python (modules décrits en partie 3)
  banc_essai/                script de dégradation PowerShell + documentation des dégradations
```

En clair : les profils et la configuration ne sont **pas** dans le code : ils sont à côté, modifiables avec un simple éditeur de texte, sans recompiler.

2.2 Lancer l'application

Depuis les sources (dans la machine virtuelle ou sur un poste de test) :

```
python src\main.py
```

Le fichier `main.py` charge la configuration, détecte les droits d'administrateur puis ouvre la fenêtre. Si `recette.ini` est absent ou incomplet, un message d'erreur en clair s'affiche et le programme s'arrête proprement.

2.3 Fichiers que l'application crée

Fichier	Dossier (venant de la config)	Rôle
`journal_AAAA-MM-JJ_HHMMSS.log`	journal	1 fichier par lancement : sondes en erreur, chaque correction (AVANT / APRÈS / durée), relectures, PV édités
`session_en_cours.json`	journal	Sauvegarde de la session (passe initiale + corrections) pour survivre à un redémarrage. Supprimé quand le PV est édité ou la session abandonnée
`PV_PC-EXP-042_2026-10-25_1450.pdf`	pv	Procès-verbal. Si 2 PV tombent dans la même minute, le 2e reçoit le suffixe _2 (on n'écrase jamais un PV)

À retenir : le chemin des dossiers dans `recette.ini` est **relatif au dossier de l'application** : racine du projet quand on lance le script, dossier de l'exécutable quand on lance l'application empaquetée (fonction `configuration.dossier_application`, qui teste `sys.frozen`).

3. Architecture générale

3.1 Les modules et leur rôle

En clair : le code est découpé en petits fichiers qui ont chacun un seul métier. Les fichiers qui parlent à Windows (sondes, corrections) ne savent pas dessiner de fenêtre ; le fichier qui dessine (interface) ne lance aucune commande système. Cette séparation est imposée par le cahier des charges (§3.4.1).

Fichier	Rôle	Connaît Tkinter ?	Appelle Windows ?
`main.py`	Point d'entrée : charge la configuration, détecte l'admin, ouvre la fenêtre	Seulement pour un message d'erreur	Détecte l'admin (ctypes)
`configuration.py`	Lit et valide `recette.ini`	Non	Non
`profil.py`	Charge et valide le profil JSON, calcule l'empreinte SHA-256	Non	Non
`catalogue.py`	Les 25 contrôles : domaine, libellé, gravité, mode de correction, consigne manuelle	Non	Non
`powershell.py`	**Fonction unique** d'appel à PowerShell	Non	Oui (c'est son métier)
`sondes.py`	1 sonde par contrôle : lit une valeur (ne juge pas, ne corrige pas)	Non	Oui (lecture)
`comparaison.py`	Compare attendu / constaté, produit les résultats et le verdict	Non	Non (appelle les sondes)
`corrections.py`	Les 14 corrections automatiques	Non	Oui (écriture)
`session.py`	Passes initiale et finale, corrections appliquées, fichier de reprise	Non	Non
`journal.py`	Crée le journal de la session (bibliothèque `logging`)	Non	Non
`systeme.py`	Identité du poste pour le PV, ouverture d'un dossier / fichier	Non	Oui
`pv.py`	Génère le PV en PDF avec fpdf2	Non	Non
`interface.py`	La fenêtre, les menus, les threads de travail	**Oui**	Non (passe par les autres modules)

3.2 Qui appelle qui

```
main.py --> configuration.py
        --> interface.py --> profil.py, catalogue.py, journal.py, session.py
                         --> comparaison.py --> sondes.py --> powershell.py
                         --> corrections.py --> comparaison.py (evaluer_controle), sondes.py, powershell.py
                         --> pv.py --> comparaison.py (calculer_verdict), catalogue.py
                         --> systeme.py --> powershell.py
```

3.3 Les trois grands scénarios

Une recette complète (bouton « Lancer la recette »)

- L'interface relit le profil (`charger_profil`) : un profil invalide affiche un message et la recette ne démarre pas.
- Elle lance un **thread de travail** qui appelle `evaluer_profil` : pour chaque contrôle du catalogue, `evaluer_controle` exécute la sonde puis le comparateur.
- Après chaque contrôle, le thread dépose un message dans la **file** ; le thread principal le lit toutes les 100 ms et met la ligne à jour (couleur, compteurs, barre).
- À la fin, si c'est la première recette complète de la session, elle devient la **passe initiale** et la session est sauvegardée sur disque.

Une correction (bouton « Corriger »)

- Si le contrôle demande une saisie (renommage : numéro du poste), une boîte de dialogue la demande.
- Une confirmation décrit en français ce qui va être fait (`corrections.confirmation`). Sans « Oui », rien n'est modifié (RG01).

- Un thread lance `corrections.appliquer` ; le résultat (succès / échec / en attente) revient par la file ; la ligne reflète la **relecture** du contrôle (RG06).
- La correction est ajoutée à la session et la session est resauvegardée immédiatement (utile avant un redémarrage).

Le procès-verbal (menu Recette > Éditer le procès-verbal)

- Actif seulement si une passe initiale existe, que la passe finale est complète (25 résultats) et que rien ne tourne.
- Refusé si l'empreinte du profil a changé entre les deux passes (les passes ne seraient plus comparables).
- L'interface copie ses données dans un `ContextePV` puis un thread appelle `generer_pv`. Le chemin du PDF s'affiche avec un bouton « Ouvrir ».

4. Les modules en détail

4.1 main.py

En clair : la porte d'entrée. Elle charge la configuration, regarde si le programme tourne en administrateur, puis ouvre la fenêtre.

Élément	Rôle
`est_administrateur()`	Appelle `ctypes.windll.shell32.IsUserAnAdmin()` ; renvoie False si l'appel échoue
`afficher_erreur(message)`	Crée une fenêtre Tk cachée uniquement pour afficher une boîte d'erreur (utile quand la fenêtre principale n'existe pas encore)
`main()`	Charge la config ; sur `ErreurConfiguration` affiche le message et quitte (code 1) ; sinon lance `Application(config, admin).lancer()`

4.2 configuration.py

En clair : lit le fichier `recette.ini` et en fait un objet `Configuration` facile à utiliser dans le reste du code.

Clé du .ini	Section	Effet
`profils`	[dossiers]	Dossier des profils JSON (relatif au dossier de l'application)
`pv`	[dossiers]	Dossier des procès-verbaux
`journal`	[dossiers]	Dossier du journal et du fichier de reprise
`cles_recuperation`	[bitlocker]	Emplacement de dépôt des clés de récupération BitLocker (chemin tel quel, pas relatif)
`defaut`	[operateur]	Nom pré-rempli dans le bandeau
`delai_max_secondes`	[commandes]	Délai maximal d'une commande PowerShell (entier > 0)

Les 6 clés sont **obligatoires** et non vides. Les erreurs (fichier absent, clé manquante, délai non entier ou <= 0) lèvent `ErreurConfiguration` avec un message en clair. Le menu « Fichier > Recharger la configuration » rappelle `charger_configuration()`.

4.3 profil.py

En clair : le profil est la « fiche de ce qu'un poste doit être ». Ce module lit le fichier JSON, vérifie qu'il est correct et calcule son empreinte (SHA-256) pour prouver, dans le PV, que le profil n'a pas été modifié après coup.

Élément	Rôle
`FORMAT`	Dictionnaire qui **décrit le format du profil** : domaine, contrôle, clé, type attendu. C'est lui qui pilote la validation
`FACULTATIVES`	Clés optionnelles (aujourd'hui seulement `chemin_reseau` de l'imprimante)
`CONTROLES_DESACTIVABLES`	Les 3 seuls contrôles qu'un profil peut désactiver : `MAT_SMART`, `MAT_TYPE_SUPPORT`, `SYS_TURN_ON`
`charger_profil(chemin)`	Lit le JSON (erreurs de syntaxe : fichier, ligne, colonne), valide, renvoie un objet `Profil`
`_valider`	Vérifie nom / version / date, chaque bloc, chaque contrôle, le booléen `actif`, chaque clé et son type
`_verifier_valeur`	Contrôle de type : texte non vide, regex compilable, nombre >= 0, entier, booléen, liste de textes, liste de logiciels, liste de services
`calculer_empreinte`	SHA-256 des octets du fichier : le moindre caractère changé donne une empreinte différente
`lister_profils(dossier)`	Les `*.json` du dossier, triés

À retenir : une valeur texte égale à `A_COMPLETER` est refusée : c'est le marqueur d'un profil non terminé. Un contrôle non désactivable avec `actif: false` est refusé. Les clés inconnues ajoutées dans le JSON sont **ignorées** sans erreur (seules les clés de `FORMAT` sont lues).

4.4 catalogue.py

En clair : la liste officielle des 25 contrôles. Chaque contrôle a un identifiant stable (par exemple `SEC_PROFILS`), un libellé lisible, un domaine, une gravité et un mode de correction. Ce n'est pas le profil : le catalogue est imposé et commun à tous les profils.

Chaque entrée est un dictionnaire `{domaine, libelle, gravite, correction, consigne}`. `DOMAINES` donne l'ordre des 7 onglets. Les constantes `BLOQUANT / MAJEUR / MINEUR` et `AUTO / MANUELLE` sont utilisées partout. La table complète est en partie 6.

4.5 powershell.py : la fonction unique

En clair : PowerShell est la façon de poser des questions à Windows et de lui donner des ordres. Au lieu de l'appeler à 40 endroits, tout passe par **une seule fonction** : `executer_powershell(commande, delai_secondes)`. Si un réglage doit changer, il change à un seul endroit.

La fonction construit un petit script qui enveloppe la commande, lance `powershell.exe -NoProfile -NonInteractive`, puis décode le JSON renvoyé. Les **4 protections** :

#	Protection	Comment	Pourquoi
1	Pas de fenêtre console	`creationflags=CREATE_NO_WINDOW`	Sans cela, une fenêtre noire clignote à chaque commande
2	Délai maximal	`timeout=delai_secondes`	Une commande bloquée ne doit pas figer la recette : elle lève `ErreurPowerShell` puis le contrôle devient « Indéterminé »
3	Encodage forcé	`[Console]::OutputEncoding = UTF8` et `encoding="utf-8"`	Évite les accents déformés (Windows utilise sinon un encodage console local)
4	Résultat en JSON	`| ConvertTo-Json -Depth 5 -Compress`	Python reçoit des données structurées (dictionnaires, listes) au lieu de texte à découper

Autres détails : `$ErrorActionPreference = 'Stop'` transforme toute erreur PowerShell en échec (code retour non nul) ; la fonction renvoie `None` si la commande ne produit aucune sortie ; erreurs possibles : délai dépassé, PowerShell introuvable, code retour non nul, JSON illisible.

À retenir : PowerShell renvoie un **texte seul** (pas une liste) quand il n'y a qu'un élément. C'est pourquoi `sondes._en_liste` existe : elle ramène toujours à une liste.

4.6 sondes.py

En clair : une sonde est un thermomètre : elle lit une valeur et la rend telle quelle. Elle ne dit jamais « c'est bon ou pas » (c'est le travail de la comparaison) et ne modifie rien.

Signature commune : `sonde_xxx(parametres, delai)` renvoie la valeur lue ou lève `ErreurSonde` (le contrôle devient « Indéterminé »). Le dictionnaire `SONDES` associe chaque identifiant du catalogue à sa fonction.

Sonde	Contrôle	Source	Valeur renvoyée
`sonde_nom_machine`	IDENT_MACHINE	Python (`socket.gethostname`)	Nom du poste (texte)
`sonde_compte_admin`	IDENT_COMPTE_ADMIN	PowerShell `Get-LocalUser`	{existe, actif}
`sonde_comptes_locaux`	IDENT_COMPTE_LOCAL	PowerShell	Liste des comptes locaux **activés**
`sonde_ram`	MAT_RAM	WMI/CIM `Win32_ComputerSystem`	Go (arrondi à 2 décimales)
`sonde_stockage`	MAT_STOCKAGE	Python (`shutil.disk_usage`)	Go libres du lecteur système
`sonde_smart / sonde_type_support`	MAT_SMART / MAT_TYPE_SUPPORT	PowerShell `Get-PhysicalDisk`	Liste (1 valeur par disque)
`sonde_os`	SYS_OS_VERSION	CIM `Win32_OperatingSystem`	{edition (Caption), version}
`sonde_activation`	SYS_TURN_ON	CIM `SoftwareLicensingProduct`	True si LicenseStatus = 1
`sonde_heure`	SYS_SERV_TIME	Service w32time, registre, `w32tm /stripchart`	{source, écart en secondes}

Sonde	Contrôle	Source	Valeur renvoyée
`sonde_ip`	SYS_IP	PowerShell `Get-NetIPInterface`	Liste « Enabled / Disabled » (DHCP) par interface connectée
`sonde_antivirus / sonde_signatures`	SEC_ANTIVIRUS / SEC_SIGN	SecurityCenter2, `Get-MpComputerStatus`	{produits, temps_reel} / âge en jours
`sonde_bitlocker`	SEC_BITLOCKER	`Get-BitLockerVolume`	True si protection « On » (admin requis)
`sonde_pare_feu`	SEC_PROFILS	`Get-NetFirewallProfile`	{profil: actif ?}
`sonde_service_maj`	SEC_MAJ_WINDOWS	`Get-Service`	{etat, demarrage}
`sonde_maj_en_attente`	SEC_MAJ	API COM `Microsoft.Update.Session`	Nombre de mises à jour (peut durer plusieurs secondes)
`sonde_logiciels_requis / _interdits`	LOG_WINGET / LOG_FORB	**Registre** (3 emplacements Uninstall)	Liste des noms de logiciels installés
`sonde_services`	LOG_START	`Get-Service`	{nom: {etat, demarrage} ou None}
`sonde_pave_numerique`	ENV_NUM_PAD	Registre HKCU `InitialKeyboardIndicators`	True si le bit 2 est à 1
`sonde_explorateur`	ENV_HIDDEN_FILES	Registre HKCU Explorer	{extensions_visibles, fichiers_caches_visibles, confirmation_suppression}
`sonde_chrome`	ENV_CHROME_START	Registre HKLM `Policies\Google\Chrome`	{mode, url}
`sonde_imprimante`	ENV_PRINT	`Get-Printer`	Liste des noms d'imprimantes
`sonde_evenements_critiques`	JRN_CRITICAL_EVENT	`Get-WinEvent`	Nombre d'événements critiques sur 24 h (journaux Système + Application)

- `_noms_logiciels` lit les logiciels dans **3 emplacements du registre** : machine, machine 32 bits (`WOW6432Node`) et utilisateur courant. La classe WMI `Win32_Product` est volontairement évitée : lente, et elle peut déclencher une réparation des paquets MSI.
- `_litteral(texte)` met un texte entre apostrophes pour PowerShell en doublant les apostrophes : c'est la protection contre les caractères qui casseraient la commande. À utiliser pour **toute valeur venant du profil** insérée dans une commande.
- `collecter_etat` sert au menu « Collecte seule » : lit les sondes des contrôles actifs sans rien comparer.

4.7 comparaison.py

En clair : c'est le « juge ». Il prend ce que le profil attend et ce que la sonde a lu, et décide : conforme, écart ou indéterminé. Il calcule aussi le verdict global du poste.

Les 4 états d'un résultat

État	Couleur	Quand
`Conforme`	Vert	La valeur constatée correspond à ce que le profil attend
`Écart`	Rouge	Elle ne correspond pas
`Indéterminé`	Orange	La sonde a échoué (service arrêté, délai dépassé, valeur incohérente). **Jamais compté comme conforme** (RG02)
`Non évalué`	Gris	Contrôle désactivé dans le profil (`actif: false`) : non exécuté, ne compte pas dans le verdict

Fonctionnement d'evaluer_controle

```
evaluer_controle(identifiant, parametres, delai, journal)
  1. parametres["actif"] est faux  -> Resultat "Non évalué"
  2. on appelle la sonde           -> ErreurSonde         -> "Indéterminé" (+ ligne au journal)
  3. on appelle le comparateur     -> KeyError/TypeError/ValueError -> "Indéterminé" (valeur inattendue)
  4. le comparateur renvoie (conforme, attendu, constaté)  -> "Conforme" ou "Écart"
```

Chaque contrôle a un **comparateur** dans le dictionnaire `COMPARATEURS` : une petite fonction `(parametres, valeur) -> (conforme, attendu, constate)`. Les textes `attendu` et `constate` sont ceux affichés dans le tableau et dans le PV. Quelques fabriques évitent la répétition : `_seuil_min` (RAM, disque), `_seuil_max` (signatures, mises à jour, événements), `_liste_egale` (S.M.A.R.T., type de support). Les comparaisons de noms de logiciels, de produit antivirus et d'édition Windows utilisent `_contient` (recherche sans tenir compte de la casse).

La classe `Resultat` (identifiant, attendu, constaté, état, message) est le modèle commun à toutes les lignes ; elle est aussi ce que `session.py` écrit dans le fichier de reprise.

4.8 corrections.py

En clair : ce module contient les actions qui modifient le poste. Elles ne partent jamais seules : seule l'interface les appelle, après le clic et la confirmation de l'opérateur. Chacune suit le même déroulement, garanti par une fonction commune, `_executer`.

Le déroulement commun (_executer)

```
1. relire le contrôle : déjà conforme ?  -> message "rien n'a été modifié"       (RG10, idempotence)
2. journaliser ce qu'on va faire         -> ligne "AVANT correction"            (RG06)
3. agir()                                -> OSError / ErreurPowerShell = ÉCHEC, avec le message de Windows
4. relire le contrôle avec la sonde      -> c'est ce résultat qui décide        (RG06)
5. journaliser le résultat et la durée   -> ligne "APRÈS correction en x s"
```

Résultat renvoyé : un `ResultatCorrection` avec un **statut** (`Succès`, `Échec`, `En attente de prise en compte`), un message lisible, un motif et la relecture. Le paramètre `attente` dit qu'après la correction, la valeur est écrite mais pas encore active (RG07) ; `declenche=True` dit que le geste lance un travail qui finira plus tard (renommage, installation de mises à jour) : la relecture ne peut pas encore être conforme et le statut est « en attente ».

Chaque correction existe en **deux fonctions** : `decrire_xxx(parametres)` renvoie le texte de la confirmation (et la « suite » : redémarrage, nouvelle session), `corriger_xxx(parametres, delai, journal)` fait le travail. Le dictionnaire `CORRECTIONS` les associe ; l'interface n'appelle que `confirmation()` et `appliquer()`.

À retenir : toute valeur du profil insérée dans une commande PowerShell passe par `_litteral()`. Le message d'erreur de Windows est restitué **tel quel** à l'opérateur (exigence F8).

4.9 session.py

En clair : la session garde en mémoire « l'avant » (passe initiale) et la liste de ce qui a été corrigé, pour pouvoir les comparer à « l'après » dans le PV. Comme certaines corrections demandent un redémarrage, la session est écrite dans un fichier pour ne pas être perdue.

Élément	Rôle
`Session`	Dataclass : début, profil (fichier, nom, version, empreinte), passe initiale (dict identifiant -> Resultat), liste des corrections
`CorrectionAppliquee`	Identifiant, statut, message, motif, heure
`enregistrer_initiale(resultats, profil)`	Copie la passe et retient quel profil l'a produite (nom, version, empreinte)
`sauvegarder(dossier)`	Écrit `session_en_cours.json` de façon **atomique** : fichier temporaire puis `replace()`. Un redémarrage en plein milieu laisse l'ancien fichier intact
`charger(dossier)`	Renvoie la session, `None` s'il n'y a pas de fichier, ou lève `ErreurSession` (format inconnu, contrôle inconnu, passe initiale absente, JSON cassé)
`supprimer(dossier)`	Efface le fichier de reprise (PV édité, session abandonnée, changement de profil)

Au lancement, `interface._proposer_reprise` propose de reprendre une session interrompue. La reprise est refusée si le profil a changé (empreinte différente) ou n'existe plus dans le dossier.

4.10 journal.py

Crée un fichier `journal_AAAA-MM-JJ_HHMMSS.log` par lancement avec la bibliothèque standard `logging` (logger nommé `recette`, niveau INFO, encodage UTF-8). Il lève `OSError` si le dossier n'est pas utilisable ; dans ce cas

l'interface **désactive les corrections** (sans journal, pas de correction : RG06). Le module `logging` est sûr à utiliser depuis plusieurs threads.

4.11 systeme.py

Fonction	Rôle
`lire_identite_poste(delai, journal)`	Nom, modèle, numéro de série, adresse MAC pour le PV. Une valeur illisible devient « non disponible » : le PV doit pouvoir être édité quand même
`ouvrir_dossier(dossier)`	Crée le dossier s'il manque puis l'ouvre dans l'explorateur (`os.startfile`)
`ouvrir_fichier(chemin)`	Ouvre un fichier avec le programme associé (le PDF dans le lecteur PDF)

4.12 pv.py

En clair : fabrique le procès-verbal PDF : un document archivable qui prouve l'état du poste avant et après la recette.

Élément	Rôle
`ContextePV`	Toutes les données du PV, **copiées** par le thread principal : le thread de travail ne lit jamais l'interface
`generer_pv(dossier, poste, ctx)`	Crée le PDF (A4 paysage), renvoie (chemin, verdict)
`_entete / _tableau / _conclusion`	3 blocs : identification (poste, profil + empreinte, contexte) ; tableau des 25 contrôles ; verdict, réserves, signature
`_action`	Texte de la colonne « correction appliquée ou geste manuel » selon ce qui s'est passé
`nom_fichier`	Convention `PV_PC-EXP-042_2026-10-08_1435.pdf`
`texte_pdf`	Rend un texte écrivable avec les polices de base (Latin-1) : symboles remplacés (>=, ->, apostrophes), caractère inconnu = « ? », jamais d'exception

Colonnes du tableau : Contrôle, Attendu, Constaté initial, Correction appliquée ou geste manuel, Constaté final, Statut (case colorée). Le bloc final est gardé d'un seul tenant (saut de page si nécessaire).

À retenir : les polices PDF de base ne gèrent pas tout Unicode. Tout texte écrit dans le PDF doit passer par `texte_pdf()` ; pour un jeu de caractères étendu, il faudrait embarquer une police TrueType.

4.13 interface.py

En clair : la fenêtre que voit le technicien. C'est le plus gros fichier : il dessine l'écran, réagit aux clics et orchestre les threads, mais ne lance aucune commande Windows lui-même.

Les 4 zones de la fenêtre

Zone	Contenu	Méthode de création
Bandeau (haut)	Nom du poste, liste des profils, opérateur, voyant des droits admin, bouton « Lancer la recette »	`_creer_bandeau`
Onglets	« Synthèse » + 1 onglet par domaine ; même tableau Contrôle / Attendu / Constaté / Statut / Correction ; le titre porte le nombre d'écarts	`_creer_onglets`, `_maj_titres`
Panneau de détail	Libellé, message, consigne « Ce qu'il faut faire », boutons « Corriger » et « Relire ce contrôle »	`_creer_detail`, `_rafraichir_detail`
Barre d'état (bas)	Barre d'avancement, compteurs, verdict global	`_creer_barre_etat`

Le menu (Fichier, Recette, Affichage, Aide) est créé par `_creer_menu`. Le statut est écrit en toutes lettres **et** coloré (lisible par un daltonien) ; les couleurs sont dans `COULEURS`.

Variables d'état importantes de la classe Application

Variable	Signification
`profil`	Dernier profil chargé et validé
`resultats`	identifiant -> Resultat de la passe en cours (vidé à chaque nouvelle recette)

Variable	Signification
`file`	`queue.Queue` : boîte aux lettres entre les threads de travail et le thread principal
`en_cours / corrections_en_cours / collecte_en_cours / pv_en_cours`	Indiquent ce qui tourne. `_occupe()` les regroupe : tant qu'une tâche tourne, les autres actions sont refusées et « Quitter » est refusé
`attentes`	identifiant -> motif de prise en compte (RG07) : la ligne reste rouge tant que l'écart persiste
`session`	La session de recette (passe initiale + corrections)
`journal_ok`	Faux si le journal n'a pas pu être créé : corrections désactivées

Quand le bouton « Corriger » est-il actif ?

Quand le contrôle est en mode Automatique **et** en Écart, qu'une fonction existe dans `CORRECTIONS`, que le contrôle n'est pas « en attente de prise en compte », que rien ne tourne et que le journal est disponible (`_rafraichir_detail`). Pour une correction manuelle, il n'y a pas de bouton : la consigne s'affiche, le technicien fait le geste puis clique « Relire ce contrôle ».

5. Le thread de travail et la file de messages

En clair : une recette peut durer plusieurs minutes (une installation winget dure 1 minute). Si Tkinter faisait ce travail lui-même, la fenêtre serait gelée : impossible de la déplacer ou de changer d'onglet. On confie donc le travail long à un second « fil d'exécution » (thread). Mais Tkinter n'aime pas qu'un autre thread touche à l'écran. Solution : le thread de travail ne dessine rien, il dépose des messages dans une boîte aux lettres (la file), et le thread principal les lit régulièrement pour mettre l'écran à jour.

```
THREAD PRINCIPAL (Tkinter)                    THREAD DE TRAVAIL
--------------------------                    -----------------
lancer_recette()  ---- threading.Thread ---->  _travail(profil)
                                                 evaluer_profil(...)   (sondes, comparaisons)
                                                 self.file.put(("resultat", n, r))
_relever_file()   <---- self.file (Queue) -----  self.file.put(("fin", None, None))
 toutes les 100 ms (fenetre.after)
 -> _afficher_resultat(), _terminer_recette()
```

Où est la frontière dans le code

- Dans `interface.py`, le commentaire **« Threads de travail : NE TOUCHENT JAMAIS AUX COMPOSANTS GRAPHIQUES »** ouvre le bloc des méthodes `_travail*` (elles ne font que calculer et appeler `self.file.put`).
- Le commentaire **« FRONTIÈRE : à partir d'ici, thread principal seulement »** ouvre le bloc qui commence par `_relever_file` : seul endroit où les messages de la file sont lus et où l'écran est modifié.
- La boucle `_relever_file` se reprogramme avec `fenetre.after(100, ...)` tant que `_occupe()` est vrai ; une seule boucle à la fois (`boucle_active`).

Les messages de la file

Type	Envoyé par	Effet côté interface
`resultat`	`_travail`	Ajoute la ligne, avance la barre, met à jour compteurs et titres des onglets
`erreur`	`_travail`	Boîte d'erreur « Erreur pendant la recette »
`fin`	`_travail`	Réactive les boutons, calcule le verdict, enregistre la passe initiale si c'est la 1re recette complète
`correction`	`_travail_correction`	Enregistre la correction dans la session, affiche la relecture (ou « en attente »), informe l'opérateur
`relecture`	`_travail_relecture`, `_travail_rejeu`	Met à jour une ligne avec la relecture de la sonde
`avancement`	`_travail_collecte`	Fait avancer la barre pendant la collecte seule
`collecte`	`_travail_collecte`	Ouvre la fenêtre secondaire avec l'état brut (texte copiable)
`pv`	`_travail_pv`	Affiche la fenêtre « Procès-verbal édité » (chemin + bouton Ouvrir) ou l'erreur

À retenir : chaque fonction `_travail*` est entourée d'un `try/except Exception` : le thread ne doit jamais mourir en silence, sinon l'interface resterait bloquée en « occupé ». Pour le PV, l'interface **copie** les données dans un `ContextePV` avant de lancer le thread, pour que le thread ne lise jamais l'interface.

6. Le format du profil de référence

En clair : le profil est un fichier JSON : c'est lui, et lui seul, qui dit ce qu'un poste doit être. Si le cabinet change de logiciel ou de seuil, on modifie ce fichier, jamais le code.

Structure générale : trois clés d'en-tête (`nom`, `version`, `date_validation`) puis un bloc par domaine. Chaque contrôle porte `actif` (true / false) et ses clés propres. Exemple extrait de `POSTE-COMPTABLE.json` :

```
{
  "nom": "POSTE-COMPTABLE", "version": "1.0", "date_validation": "2026-10-11",
  "securite": {
    "SEC_SIGN":  {"actif": true, "age_max_jours": 3},
    "SEC_MAJ":   {"actif": true, "maximum": 2}
  }
}
```

En-tête (obligatoire, texte non vide)

Clé	Type	Effet
`nom`	texte	Nom du profil, affiché dans « À propos » et imprimé dans le PV
`version`	texte	Version du profil, imprimée dans le PV
`date_validation`	texte	Date de validation du profil, imprimée dans le PV

Toutes les clés des contrôles (générées depuis profil.FORMAT)

Contrôle	Clé	Type	Oblig.	Effet sur la recette
`IDENT_MACHINE`	`actif`	booléen	oui	doit rester true
`IDENT_MACHINE`	`prefixe`	texte non vide	oui	Préfixe du nom (ex. PC-EXP) ; sert à construire le nom lors du renommage
`IDENT_MACHINE`	`modele`	expression régulière	oui	Expression régulière que le nom de la machine doit respecter ; valide aussi le nom saisi
`IDENT_COMPTE_ADMIN`	`actif`	booléen	oui	doit rester true
`IDENT_COMPTE_ADMIN`	`compte`	texte non vide	oui	Nom du compte d'administration technique attendu (présent et actif)
`IDENT_COMPTE_LOCAL`	`actif`	booléen	oui	doit rester true
`IDENT_COMPTE_LOCAL`	`comptes_autorises`	liste de textes	oui	Seuls comptes locaux activés autorisés ; les autres sont des écarts (et désactivables)
`MAT_RAM`	`actif`	booléen	oui	doit rester true
`MAT_RAM`	`minimum_go`	nombre >= 0	oui	Mémoire minimale, en Go
`MAT_STOCKAGE`	`actif`	booléen	oui	doit rester true
`MAT_STOCKAGE`	`libre_minimum_go`	nombre >= 0	oui	Espace libre minimal du lecteur système, en Go
`MAT_SMART`	`actif`	booléen	oui	false = non évalué (autorisé seulement pour MAT_SMART, MAT_TYPE_SUPPORT, SYS_TURN_ON)
`MAT_SMART`	`sante_attendue`	texte non vide	oui	Santé attendue de chaque disque (ex. Healthy)
`MAT_TYPE_SUPPORT`	`actif`	booléen	oui	false = non évalué (autorisé seulement pour MAT_SMART, MAT_TYPE_SUPPORT, SYS_TURN_ON)
`MAT_TYPE_SUPPORT`	`type_attendu`	texte non vide	oui	Type de support attendu (ex. SSD)
`SYS_OS_VERSION`	`actif`	booléen	oui	doit rester true
`SYS_OS_VERSION`	`edition`	texte non vide	oui	Texte recherché dans le nom de l'édition Windows (dépend de la langue de Windows)

Contrôle	Clé	Type	Oblig.	Effet sur la recette
SYS_OS_VERSION	version	texte non vide	oui	Numéro de version exact attendu (ex. 10.0.26200)
SYS_TURN_ON	actif	booléen	oui	false = non évalué (autorisé seulement pour MAT_SMART, MAT_TYPE_SUPPORT, SYS_TURN_ON)
SYS_TURN_ON	activation_exigee	true / false	oui	Si true, Windows doit être activé
SYS_SERV_TIME	actif	booléen	oui	doit rester true
SYS_SERV_TIME	serveur	texte non vide	oui	Serveur de temps attendu ; doit apparaître dans la configuration NTP du poste ; sert aussi à la correction
SYS_SERV_TIME	ecart_max_secondes	nombre >= 0	oui	Écart d'horloge maximal toléré, en secondes
SYS_IP	actif	booléen	oui	doit rester true
SYS_IP	dhcp	true / false	oui	Si true, toutes les interfaces connectées doivent être en DHCP
SEC_ANTIVIRUS	actif	booléen	oui	doit rester true
SEC_ANTIVIRUS	produit	texte non vide	oui	Texte recherché dans les antivirus déclarés (temps réel actif exigé)
SEC_SIGN	actif	booléen	oui	doit rester true
SEC_SIGN	age_max_jours	entier >= 0	oui	Âge maximal des signatures antivirus, en jours
SEC_BITLOCKER	actif	booléen	oui	doit rester true
SEC_BITLOCKER	exige	true / false	oui	Si true, la protection BitLocker du lecteur système doit être activée
SEC_PROFILS	actif	booléen	oui	doit rester true
SEC_PROFILS	profils	liste de textes	oui	Profils du pare-feu qui doivent être actifs (Domain, Private, Public)
SEC_MAJ_WINDOWS	actif	booléen	oui	doit rester true
SEC_MAJ_WINDOWS	service	texte non vide	oui	Nom du service de mise à jour de Windows (wuauserv)
SEC_MAJ	actif	booléen	oui	doit rester true
SEC_MAJ	maximum	entier >= 0	oui	Nombre maximal de mises à jour en attente
LOG_WINGET	actif	booléen	oui	doit rester true
LOG_WINGET	requis	liste d'objets {nom, winget_id}	oui	Logiciels du socle, avec leur identifiant winget (utilisé pour l'installation)
LOG_FORB	actif	booléen	oui	doit rester true
LOG_FORB	interdits	liste de textes	oui	Textes recherchés dans les logiciels installés : tout logiciel trouvé est un écart
LOG_START	actif	booléen	oui	doit rester true
LOG_START	services	liste d'objets {nom, etat, demarrage}	oui	Services avec état (Running) et type de démarrage (Automatic) attendus
ENV_NUM_PAD	actif	booléen	oui	doit rester true
ENV_NUM_PAD	attendu	true / false	oui	true : pavé numérique actif au démarrage ; false : inactif
ENV_HIDDEN_FILES	actif	booléen	oui	doit rester true
ENV_HIDDEN_FILES	extensions_visibles	true / false	oui	Extensions de fichiers visibles dans l'explorateur
ENV_HIDDEN_FILES	fichiers_caches_visibles	true / false	oui	Fichiers cachés visibles
ENV_HIDDEN_FILES	confirmation_suppression	true / false	oui	Confirmation de suppression active
ENV_CHROME_START	actif	booléen	oui	doit rester true

Contrôle	Clé	Type	Oblig.	Effet sur la recette
ENV_CHROME_START	url	texte non vide	oui	Page de démarrage de Chrome imposée par stratégie
ENV_PRINT	actif	booléen	oui	doit rester true
ENV_PRINT	nom	texte non vide	oui	Nom de l'imprimante attendue
JRN_CRITICAL_EVENT	actif	booléen	oui	doit rester true
JRN_CRITICAL_EVENT	maximum_24h	entier >= 0	oui	Nombre maximal d'événements critiques sur 24 h
ENV_PRINT	chemin_reseau	texte non vide	non (défaut : absent)	Facultatif : chemin réseau de l'imprimante, affiché dans la consigne manuelle

Les valeurs de dégradation du banc d'essai ne sont pas dans les profils : elles sont écrites dans le script PowerShell `banc_essai/degradation.ps1` (choix documenté du projet).

Les trois profils fournis

Profil	RAM min.	Disque libre min.	Serveur de temps	MAJ max.	Particularités
POSTE-COMPTABLE	3,5 Go	3 Go	time.cloudflare.com (5 s)	2	Profil du banc d'essai : MAT_SMART, MAT_TYPE_SUPPORT et SYS_TURN_ON désactivés ; édition « Entreprise » 10.0.26200 ; services Spooler + Audiosrv
POSTE-ACCUEIL	8 Go	30 Go	time.windows.com (10 s)	5	Édition Pro 10.0.26100 ; imprimante Imprimante-Accueil ; 3 logiciels du socle
POSTE-DIRECTION	16 Go	100 Go	time.windows.com (2 s)	2	Pavé numérique inactif ; signatures <= 1 jour ; imprimante Imprimante-Direction ; service W32Time

À retenir : POSTE-ACCUEIL et POSTE-DIRECTION servent à démontrer que le profil pilote la recette sans modifier le code ; ils ne cherchent pas à rendre la machine du banc d'essai conforme.

7. Le catalogue des 25 contrôles

En clair : pour chaque contrôle : sa gravité (qui détermine le verdict) et ce que fait l'application devant un écart. « Automatique » = bouton « Corriger » ; « Manuelle » = consigne affichée au technicien.

Identifiant	Domaine	Contrôle	Gravité	Correction	Consigne (si manuelle)
IDENT_MACHINE	Identité	Nom de la machine conforme à la convention	Majeur	Automatique	-
IDENT_COMPTE_ADMIN	Identité	Compte d'administration technique présent et actif	Bloquant	Manuelle	Créer le compte local d'administration technique défini dans le profil (« Gestion de l'ordinateur > Utilisateurs et groupes locaux »), l'ajouter au groupe Administrateurs, vérifier qu'il est activé, puis lui attribuer le mot de passe du service.
IDENT_COMPTE_LOCAL	Identité	Aucun compte local en dehors de la liste du profil	Bloquant	Automatique	-
MAT_RAM	Matériel	Mémoire vive au moins égale au seuil	Majeur	Manuelle	Rajouter de la mémoire vive afin d'atteindre le seuil requis
MAT_STOCKAGE	Matériel	Espace disque libre au moins égal au seuil	Majeur	Manuelle	Rajouter du stockage afin d'atteindre le seuil requis
MAT_SMART	Matériel	Santé S.M.A.R.T. du disque correcte	Bloquant	Manuelle	Suite à l'analyse SMART de votre disque, renvoyer le poste au stock
MAT_TYPE_SUPPORT	Matériel	Type de support identifié et entretien adapté effectué	Mineur	Automatique	-
SYS_OS_VERSION	Système	Édition et version de Windows attendues	Bloquant	Manuelle	Merci d'installer et d'utiliser la bonne édition et version de Windows
SYS_TURN_ON	Système	État d'activation conforme à l'exigence du profil	Majeur	Manuelle	Activer Windows avec une licence valide (« Paramètres > Système > Activation »)
SYS_SERV_TIME	Système	Heure synchronisée avec le serveur de temps, écart sous le seuil	Majeur	Automatique	-
SYS_IP	Système	Adresse IP obtenue par DHCP, aucune adresse fixe résiduelle	Majeur	Automatique	-
SEC_ANTIVIRUS	Sécurité	Antivirus actif, protection en temps réel	Bloquant	Manuelle	Veuillez activer votre antivirus
SEC_SIGN	Sécurité	Signatures antivirus plus récentes que le seuil	Majeur	Automatique	-
SEC_BITLOCKER	Sécurité	État de BitLocker conforme à l'exigence du profil	Bloquant	Manuelle	Activer BitLocker sur le disque système, puis déposer la clé de récupération à l'emplacement indiqué dans la configuration
SEC_PROFILS	Sécurité	Les 3 profils du pare-feu actifs	Bloquant	Automatique	-
SEC_MAJ_WINDOWS	Sécurité	Service de mise à jour de Windows actif et démarrable	Bloquant	Automatique	-
SEC_MAJ	Sécurité	Mises à jour en attente sous le seuil	Majeur	Automatique	-
LOG_WINGET	Logiciels	Chaque logiciel du socle géré par winget présent	Majeur	Automatique	-
LOG_FORB	Logiciels	Aucun logiciel interdit	Majeur	Manuelle	Désinstaller le logiciel interdit signalé par l'application

Identifiant	Domaine	Contrôle	Gravité	Correction	Consigne (si manuelle)
`LOG_START`	Logiciels	Services requis démarrés, en démarrage automatique	Mineur	Automatique	-
`ENV_NUM_PAD`	Environnement	Pavé numérique actif au démarrage	Mineur	Automatique	-
`ENV_HIDDEN_FILES`	Environnement	Extensions et fichiers cachés visibles, confirmation de suppression active	Mineur	Automatique	-
`ENV_CHROME_START`	Environnement	Page de démarrage de Chrome imposée par stratégie	Mineur	Automatique	-
`ENV_PRINT`	Environnement	Imprimante du service présente	Majeur	Manuelle	Ajouter l'imprimante dont le nom est affiché, avec son chemin réseau s'il est défini dans le profil
`JRN_CRITICAL_EVENT`	Journaux	Nombre d'événements critiques sur les dernières 24 h au plus égal au seuil	Mineur	Manuelle	Consulter la liste affichée des événements critiques des dernières 24 h, en identifier la cause et la corriger

Total : 25 contrôles, dont 14 à correction automatique et 11 à correction manuelle.

8. La règle du verdict (RG03)

En clair : le verdict global dépend de la **gravité** des écarts, pas de leur nombre. La fonction `calculer_verdict` applique ces 4 cas dans l'ordre.

Situation (contrôles actifs seulement)	Verdict	Motif affiché
Aucun écart et aucun indéterminé	Conforme	-
Uniquement des écarts **mineurs**	Conforme avec réserves	Les écarts mineurs sont listés comme « réserves » dans le PV
Au moins 1 écart **majeur** ou **bloquant**	Non conforme	écart majeur ou bloquant
Au moins 1 contrôle **indéterminé**	Non conforme	contrôle non vérifié (on ne valide pas ce qu'on n'a pas pu lire)

Les contrôles « Non évalués » (désactivés dans le profil) n'entrent pas dans le calcul. Les deux motifs peuvent se cumuler.

À retenir : un contrôle « en attente de prise en compte » (RG07) reste affiché en **écart** : il pèse donc dans le verdict tant que le redémarrage ou la nouvelle session n'a pas eu lieu et que le contrôle n'a pas été relu.

9. Les 14 corrections automatiques

Contrôle	Ce que fait la correction	Droits admin	Prise en compte
`IDENT_MACHINE`	`Rename-Computer` avec préfixe du profil + numéro saisi, validé par la regex du profil	Oui	Redémarrage (en attente)
`IDENT_COMPTE_LOCAL`	`Disable-LocalUser` pour chaque compte activé hors liste. Jamais de suppression (RG08). Refuse de désactiver le compte de la session en cours	Oui	Immédiate
`MAT_TYPE_SUPPORT`	Détecte le support du disque système : HDD = analyse + défragmentation ; SSD = TRIM (`Optimize-Volume`). Type inconnu : refus, rien n'est lancé	Oui	Immédiate
`SYS_SERV_TIME`	`w32tm /config /manualpeerlist` (serveur du profil) puis `/resync /rediscover` ; démarre w32time si besoin	Oui	Immédiate
`SYS_IP`	DHCP activé, adresse et passerelle fixes retirées, DNS remis en automatique. La connexion peut s'interrompre quelques secondes	Oui	Immédiate
`SEC_SIGN`	`Update-MpSignature`	Oui	Immédiate
`SEC_PROFILS`	`Set-NetFirewallProfile -Enabled True` pour les profils du profil JSON	Oui	Immédiate
`SEC_MAJ_WINDOWS`	`Set-Service -StartupType Manual` : réactive le service (RG09), n'arrête jamais rien	Oui	Immédiate
`SEC_MAJ`	Lance, dans un PowerShell séparé et caché, la recherche + téléchargement + installation des mises à jour (script en base64). Ne relance pas si déjà en cours (RG10)	Oui	Installation en arrière-plan puis redémarrage
`LOG_WINGET`	Relit le registre, installe seulement les logiciels manquants par `winget install --id ... -e --silent`. Le code retour de winget n'est pas fiable : la relecture décide	Oui	Immédiate (relecture du registre)
`LOG_START`	`Set-Service -StartupType` puis `Start-Service` pour chaque service du profil (jamais d'arrêt)	Oui	Immédiate
`ENV_NUM_PAD`	Registre HKCU `InitialKeyboardIndicators` : modifie uniquement le bit 2 (les autres bits sont conservés)	Non (HKCU)	Nouvelle ouverture de session
`ENV_HIDDEN_FILES`	Registre HKCU : `HideFileExt`, `Hidden`, `ConfirmFileDelete`	Non (HKCU)	Nouvelle ouverture de session
`ENV_CHROME_START`	Registre HKLM, stratégie `RestoreOnStartup = 4` et `RestoreOnStartupURLs\1` (adresse du profil)	Oui (HKLM)	Immédiate ; relancer Chrome

Points d'attention

- **HKCU contre HKLM :** les réglages d'environnement s'écrivent dans la ruche de l'**utilisateur courant** : ils ne s'appliquent qu'à lui (pas aux futurs utilisateurs, ce qui demanderait de modifier le profil par défaut, non traité). L'explorateur ne relit pas le registre instantanément.
- **Stratégie contre préférence (Chrome) :** une clé sous `Policies` **s'impose** à l'utilisateur ; une préférence (réglage dans Chrome) pourrait être changée par lui.
- **Les 3 fonctions de l'ancien catalogue** n'ont pas été reprises telles quelles : Internet Explorer (disparu) devient la stratégie Chrome ; « arrêter Windows Update » est inversé en réactivation (RG09) ; « défragmenter le disque » devient un entretien adapté au support (inutile et usant sur un SSD).
- **Renommage :** sans inventaire central, l'application ne peut pas choisir un numéro libre : l'opérateur le saisit (`saisie_requise` + `simpledialog` dans l'interface), puis le nom complet est validé par la regex du profil.
- **Corriger ne garantit rien tant que la relecture n'est pas verte :** en cas d'écart persistant, la raison renvoyée par Windows est affichée telle quelle.

10. Faire évoluer le projet

10.1 Ajouter un profil

- Copier un profil existant dans le dossier des profils, le renommer (le nom du fichier sans `.json` apparaît dans la liste déroulante).
- Modifier `nom`, `version`, `date_validation` et les valeurs. Aucun code à toucher.
- Lancer l'application : tout défaut de format est annoncé avec le fichier et la clé concernée (RG05).

10.2 Ajouter un contrôle (ordre à suivre)

#	Fichier	À faire
1	`catalogue.py`	Ajouter l'entrée dans `CATALOGUE` (domaine, libellé, gravité, mode de correction, consigne si manuelle)
2	`profil.py`	Ajouter le contrôle et ses clés dans `FORMAT` (type de chaque clé)
3	Les JSON de `profils/`	Ajouter le bloc du contrôle (avec `actif` et ses clés) dans **tous** les profils, sinon ils seront refusés
4	`sondes.py`	Écrire `sonde_xxx(parametres, delai)` et l'ajouter à `SONDES`
5	`comparaison.py`	Écrire le comparateur et l'ajouter à `COMPARATEURS`
6	`corrections.py`	Si automatique : écrire `decrire_xxx` et `corriger_xxx` (en passant par `_executer`) puis l'ajouter à `CORRECTIONS`
7	Plan de tests	Ajouter les cas de test : état conforme, état dégradé, correction, 2e lancement

À retenir : l'onglet du domaine, le compteur d'écarts, la barre d'avancement (`maximum=len(CATALOGUE)`), le PV et le fichier de reprise (qui refuse un identifiant absent du catalogue) s'adaptent automatiquement. Un nouveau **domaine**, lui, doit aussi être ajouté à `DOMAINES` et à `FORMAT`.

10.3 Règles à respecter quand on modifie le code

- Aucun chemin, seuil, nom de logiciel, d'imprimante ou de serveur dans un fichier Python : profil ou configuration (RG04). Les noms propres à Windows (classes CIM, clés de registre) sont admis dans les sondes et corrections.
- Un appel à PowerShell passe toujours par `executer_powershell` ; toute valeur du profil insérée dans une commande passe par `_litteral`.
- Les sondes et corrections n'importent jamais Tkinter ; l'interface ne lance aucune commande système.
- Seul le thread principal touche aux composants graphiques : un thread de travail dépose un message dans la file.
- Une correction passe par `_executer` (journal avant / après, relecture, idempotence) et ne doit jamais supprimer, désinstaller ni chiffrer (RG08).
- Tester **dans la machine virtuelle**, après restauration de l'instantané voulu ; jamais sur un poste réel.

10.4 Dépannage

Symptôme	Cause probable	Que faire
« Fichier de configuration introuvable »	`recette.ini` absent à côté de l'application	Le placer à la racine du projet (ou à côté de l'exécutable)
« Profil invalide » avec fichier, ligne ou clé	Erreur de syntaxe JSON ou valeur incorrecte (RG05)	Corriger la ligne ou la clé indiquée dans le message
Aucun profil dans la liste	Dossier des profils vide ou mal indiqué dans la config	Vérifier `[dossiers] profils` ; menu Fichier > Choisir le profil...
Boutons « Corriger » grisés	Journal indisponible, ligne conforme/indéterminée, correction manuelle, ou tâche en cours	Lire le message du panneau de détail ; vérifier les droits d'écriture du dossier du journal
Contrôle « Indéterminé »	Sonde en erreur (service arrêté, délai dépassé, accès refusé)	Lire la raison affichée ; relancer en administrateur ; consulter le journal

Symptôme	Cause probable	Que faire
BitLocker « accès refusé »	Application lancée sans droits d'administrateur	Relancer en administrateur
Écart d'heure parasite sur la VM	Dérive d'horloge de la machine virtuelle	Resynchroniser l'horloge (`w32tm /resync`) ; vérifier les réglages de synchronisation de la VM
Un PDF n'est pas créé	Dossier des PV non inscriptible	Vérifier le dossier configuré et ses droits (dossier partagé de VM : décocher « lecture seule »)
Message de reprise de session au lancement	Une session a été interrompue (redémarrage)	Répondre Oui pour reprendre, puis relancer la recette pour obtenir la passe finale

11. Dépendances

Paquet	Version	Rôle
`fpdf2`	2.8.9	Génération directe du PDF du procès-verbal (`pv.py`). Seule bibliothèque dédiée au PDF
`pillow`	13.0.0	Dépendance de fpdf2
`fonttools`	4.67.0	Dépendance de fpdf2
`defusedxml`	0.7.1	Dépendance de fpdf2
`pyinstaller`	6.22.3	Empaquetage en un exécutable unique

Tout le reste vient de la bibliothèque standard de Python : `tkinter`, `json`, `configparser`, `logging`, `hashlib`, `subprocess`, `threading`, `queue`, `winreg`, `ctypes`, `re`, `dataclasses`, `pathlib`, `shutil`, `socket`, `base64`. Outils externes utilisés par les commandes : PowerShell, WMI/CIM, `winget`, `w32tm`.

À retenir : la distribution Python « embeddable » n'a pas Tkinter et n'a pas `pip`. Sur le banc d'essai, l'application tourne avec une copie du Python complet (dossier `python-tk`, non versionné) ; fpdf2 et ses dépendances y ont été installés depuis le poste de développement avec `pip install --target`.

12. Procédure d'empaquetage (à compléter)

Cette partie correspond à la phase de livraison, non terminée au moment de la rédaction. Elle sera à compléter avec la procédure réellement testée.

Ce que le projet impose : un exécutable unique produit par PyInstaller, sans console, qui ne contient **ni les profils ni la configuration** (livrés à côté, modifiables sans recompiler), testé dans la machine virtuelle **sans Python installé**. La stratégie d'élévation recommandée est la demande de droits d'administrateur au lancement, par le manifeste de l'exécutable.

Exemple de commande à adapter et à **valider dans la machine virtuelle** (options standard de PyInstaller) :

```
pyinstaller --onefile --noconsole --uac-admin --name RecettePostes src\main.py
```

À renseigner une fois testé	Valeur
Commande exacte et options utilisées	à compléter
Fichier .spec éventuel et modules cachés ajoutés	à compléter
Contenu du dossier livré (exécutable, recette.ini, profils/)	à compléter
Comportement si l'invite de contrôle de compte est refusée	à compléter
Justification de la stratégie d'élévation retenue	à compléter

13. Limites connues et points de vigilance

- **Corrections non validées sur le banc d'essai :** `SEC_SIGN` (suppression des signatures refusée par Windows dans la machine virtuelle : écart non reproductible) et `SEC_MAJ` (contrôle vert sur le banc : aucun écart à corriger). Elles sont écrites mais non testées en conditions réelles.
- **MAT_TYPE_SUPPORT :** la machine virtuelle n'expose pas le type de support ; la logique d'entretien (`commandes_entretien`) est une fonction pure testée par un cas simulé (HDD, SSD, type non détecté).

- **3 contrôles non évalués sur le banc :** `MAT_SMART`, `MAT_TYPE_SUPPORT`, `SYS_TURN_ON` (Windows renvoie « inconnu » ou la licence n'est pas activée sur une machine virtuelle). Ils restent obligatoires dans les autres profils.
- **Clé de récupération BitLocker :** le chemin `cles_recuperation` est lu dans la configuration mais n'est pas encore utilisé par le code ; la consigne BitLocker du catalogue renvoie seulement à « l'emplacement indiqué dans la configuration ».
- **Dépendance à la langue de Windows :** la comparaison d'édition utilise le texte du nom (« Entreprise », « Pro ») : il change selon la langue du système.
- **Clés inconnues du profil ignorées :** une faute de frappe dans une clé facultative ne produit pas d'erreur ; seules les clés de `profil.FORMAT` sont vérifiées.
- **Dépendances internes entre modules :** `corrections.py` importe des fonctions « privées » (préfixe _) de `sondes.py` et `comparaison.py` (`_litteral, _noms_logiciels, _contient`) : renommer ces fonctions impose de corriger les imports.
- **Sonde de mises à jour :** `SEC_MAJ` peut dépasser un délai de commande court (cas déjà rencontré à 120 s) ; le délai se règle dans `recette.ini` (`delai_max_secondes`), actuellement 300.
- **Périmètre volontairement exclu :** annuaire Active Directory, supervision continue, planification, exécution à distance, désinstallation automatique, base de données, interface web, signature électronique du PV, intégration à FOG.

14. Glossaire

Terme	Définition
Recette	Vérification formelle qu'un livrable (ici un poste) est conforme à une référence écrite, avec un verdict et une trace signée
Profil	Fichier JSON qui décrit ce qu'un type de poste doit être (valeurs attendues)
Sonde	Fonction qui lit une valeur sur le poste, sans juger ni modifier
Comparateur	Fonction qui décide si la valeur lue correspond à l'attendu
Écart / Indéterminé	Valeur non conforme / valeur impossible à lire (jamais « conforme »)
Verdict	Conclusion globale : Conforme, Conforme avec réserves, Non conforme
Passe initiale / finale	Première recette complète (avant corrections) / recette complète finale (après)
Idempotent	Qu'on peut lancer plusieurs fois avec le même résultat que de le lancer une fois
Empreinte SHA-256	« Signature » d'un fichier : change si un seul caractère du fichier change
Thread / file de messages	Fil d'exécution parallèle / boîte aux lettres par laquelle il communique avec l'interface
Registre (HKLM, HKCU)	Base de réglages de Windows : HKLM = machine entière, HKCU = utilisateur courant
WMI / CIM	Interface par laquelle Windows expose des informations sur la machine (matériel, système)
PowerShell	Langage de commande de Windows, utilisé pour lire l'état du poste et exécuter les corrections
winget	Gestionnaire de paquets de Windows, utilisé pour installer les logiciels manquants
Instantané	Photo de l'état de la machine virtuelle, restaurable en quelques secondes
PV	Procès-verbal de recette, document PDF horodaté et archivé